

PROJETO MULTIDISCIPLINAR - TRILHA BACK-END

PROJETO BACK-END

Patricky Moreno Pinheiro - RU 4929339

RAÍZES DO NORDESTE

API de pedidos multicanal, estoque e pagamento simulado

Projeto Back-End - Atividade Prática 2026

2026

SUMÁRIO

1 INTRODUÇÃO E ESCOPO

Situação da entrega

2 REQUISITOS E PRIORIZAÇÃO

Justificativa da priorização

3 ARQUITETURA E CASOS DE USO

Decisões de projeto

Feature crítica: realizar pedido e pagar

4 MODELO DE DADOS E CLASSES

5 CONTRATOS REST E REGRAS OPERACIONAIS

Paginação e códigos HTTP

Exemplo de pedido

Exemplo de pagamento

Transições, cancelamento e fidelidade

6 SEGURANÇA E PRIVACIDADE

Dados e finalidade técnica

Proposta acadêmica de bases e retenção

Implementado

Limitações explícitas

7 TESTES E RESULTADOS

Plano executável da coleção

8 EXECUÇÃO E ENTREGA

9 CONCLUSÃO E EVOLUÇÕES

REFERÊNCIAS

1 INTRODUÇÃO E ESCOPO

A rede fictícia Raízes do Nordeste necessita consolidar pedidos originados de aplicativo, totem, balcão, retirada e web. O projeto organiza as operações em uma API REST, com dados relacionais e acesso controlado por perfis. O objetivo é demonstrar um fluxo íntegro e reproduzível: criação do pedido, pagamento simulado, baixa de estoque, preparo e finalização.

O MVP adota o fluxo A do roteiro: pedido até pagamento mock e acompanhamento de status. O campo canalPedido é obrigatório e pode ser consultado como filtro. As demais funcionalidades são implementadas quando sustentam esse fluxo ou descritas como evolução conceitual. Não há integração financeira real nem coleta de dados de cartão.

A entrega técnica inclui código C#/.NET 10, migration PostgreSQL, dados fictícios iniciais, Swagger/OpenAPI, coleção Postman, testes automatizados e documentação. A execução local usa PostgreSQL. Docker constitui uma alternativa de empacotamento; hospedagem online é opcional.

Situação da entrega

A validação técnica registrou 36 testes xUnit aprovados. A coleção Postman executou 44 requisições e 79 asserções no PostgreSQL, sem falhas. O PostgreSQL foi testado nativamente; Docker permanece opcional e sua execução local depende de virtualização ativa. O repositório público respondeu sem autenticação; as alterações finais dependem de commit e push dos integrantes.

Repositório: <https://github.com/PatrickyMP/RaizesDoNordeste>

2 REQUISITOS E PRIORIZAÇÃO

A rede precisa consolidar pedidos de APP, TOTEM, BALCAO, PICKUP e WEB, respeitando o estoque de cada unidade. O MVP prioriza um fluxo de pedido íntegro e testável, sem cobrança financeira real.

ID	Requisito	Implementação/prioridade
RF01	Cadastrar e autenticar clientes	Login JWT e hash de senha; alta
RF02	Diferenciar perfis operacionais	Cliente, Atendente, Cozinha, Entregador, Gerente, Administrador; alta
RF03	Consultar cardápio por unidade	Produtos ativos com saldo disponível na unidade; alta

RF04	Criar e consultar pedido multicanal	Canal obrigatório, itens, preços do banco, tipo de atendimento; alta
RF05	Processar pagamento mock	Aprovação, recusa, indisponibilidade e idempotência; alta
RF06	Atualizar/cancelar pedido	Transições válidas por atendimento; cancelamento antes do preparo; alta
RF07	Movimentar estoque	Entradas, saídas manuais/na aprovação, histórico e estorno; alta
RF08	Fidelizar com consentimento	Pontos, resgate e revogação de consentimento; média
RF09	Promoções/campanhas	Proposta: desconto por unidade/canal, janela temporal e limite; não implementado
RF10	Gestão completa da rede	Cadastro/consulta/edição/desativação de unidades e produtos; vínculo de funcionários à unidade fica proposto
ID	Requisito não funcional	Abordagem e limitação
RNF01	Integridade	Transação serializável; concorrência otimista em User, Stock e Order; FKs e unicidade
RNF02	Segurança	JWT, perfis, minimização de dados e erros sem stack trace; não é certificação de segurança
RNF03	Rastreabilidade	Canal persistido e AuditLogs para ações sensíveis
RNF04	Reprodutibilidade	.NET 10, seed, PostgreSQL local, Docker opcional, coleção e testes
RNF05	Desempenho	Paginação e índices; sem benchmark ou garantia de capacidade em pico
RNF06	Disponibilidade	Endpoint /health, banco persistente e falha mock controlada; sem SLA de disponibilidade
RNF07	Evolução	Camadas e serviços; migration PostgreSQL versionada

Justificativa da priorização

Pedidos, autenticação, pagamento e estoque têm prioridade alta porque uma falha pode permitir acesso indevido, duplicar uma cobrança simulada ou vender quantidade inexistente. Fidelidade utiliza o mesmo fluxo transacional e tem prioridade média. Campanhas e gestão avançada por unidade não são necessárias para comprovar o fluxo A e permanecem propostas explícitas.

3 ARQUITETURA E CASOS DE USO

OrderService orquestra criação, pagamento, estoque, transições, cancelamento e pontos. IAppDbContext abstrai a unidade de trabalho; expõe DbSet, portanto a camada Application ainda conhece EF Core. É uma separação pragmática, não uma implementação estrita de

Clean Architecture. Cadastros simples permanecem nos controllers. MockPaymentGateway implementa IPaymentGateway e representa um provedor externo sem dependência de rede.

Camada	Responsabilidade
API	Rotas, validação de contratos, JWT, perfis, erros e Swagger.
Application	OrderService: criação, pagamento, estoque, status, cancelamento e pontos.
Domain	Entidades, enums e OrderRules; regras de transição e concessão de pontos.
Infrastructure	EF Core, AppDbContext, migration e MockPaymentGateway.

Decisões de projeto

Interfaces isolam o processamento do pagamento e a unidade de trabalho. A camada Application ainda conhece DbSet/EF Core, decisão pragmática para reduzir complexidade neste protótipo. Os controllers de cadastro usam o contexto diretamente. Não se apresenta essa organização como Clean Architecture estrita.

Casos de uso - Raízes do Nordeste

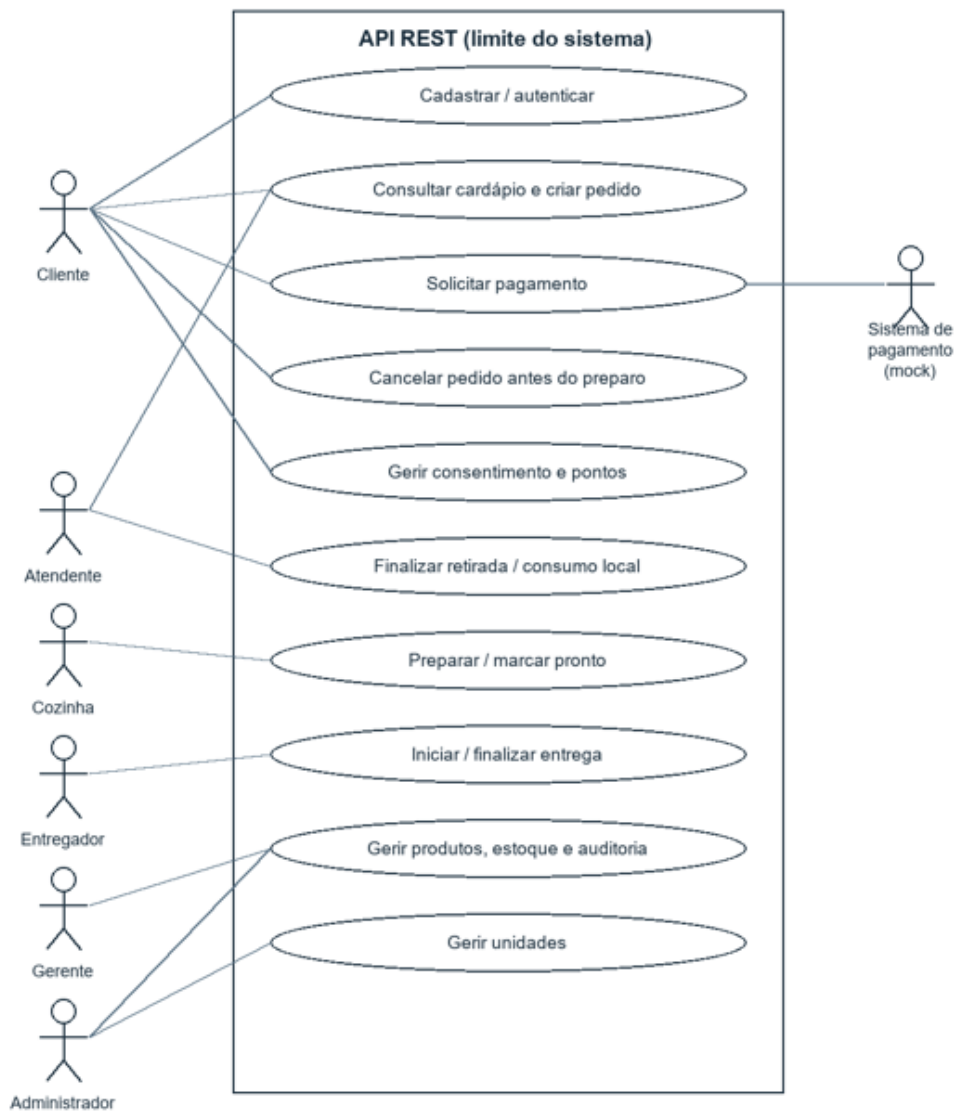


Figura 1 - Casos de uso e atores.

A figura apresenta os principais casos de uso, sem repetir todas as associações de autenticação e consulta. Gerente/Administrador também podem conduzir transições operacionais válidas. O mock representa o sistema externo na análise, mas executa como adaptador interno neste MVP.

2

Feature crítica: realizar pedido e pagar

Pré-condições: usuário autenticado; unidade e produtos ativos; quantidades válidas; estoque suficiente. Se houver resgate: consentimento e pontos suficientes. Pedido de entrega deve ter endereço.

Fluxo principal: API valida contrato e canal → carrega produtos e calcula total → verifica estoque agregado → reserva pontos de resgate → salva pedido e auditoria → usuário solicita mock com chave única → aprovação verifica novamente e baixa estoque → salva pagamento,

payload e status Aceito atomicamente → cozinha prepara e marca pronto → atendente/entregador finaliza → credita pontos uma única vez se houver consentimento.

Pós-condições: pedido finalizado, pagamentos registrados, estoque atualizado, pontos consistentes e trilha de auditoria consultável.

Exceções: contrato inválido 400/422; ausência de token 401; falta de permissão 403; unidade/produto/pedido inexistente 404; estoque insuficiente, chave reutilizada com outro payload ou transição inválida 409. Recusa do mock é resultado de negócio HTTP 200 com status=Recusado; indisponibilidade simulada é HTTP 503. O consumidor pode repetir a operação após consultar o estado; a idempotência evita duplicação de pagamento.

4 MODELO DE DADOS E CLASSES

O modelo possui nove tabelas de domínio. As chaves primárias são GUIDs; as migrations acrescentam tabelas de controle do EF Core. O preço é decimal e utiliza precisão (12,2) no PostgreSQL. Datas são geradas em UTC. Relações e restrições acompanham o modelo persistido.

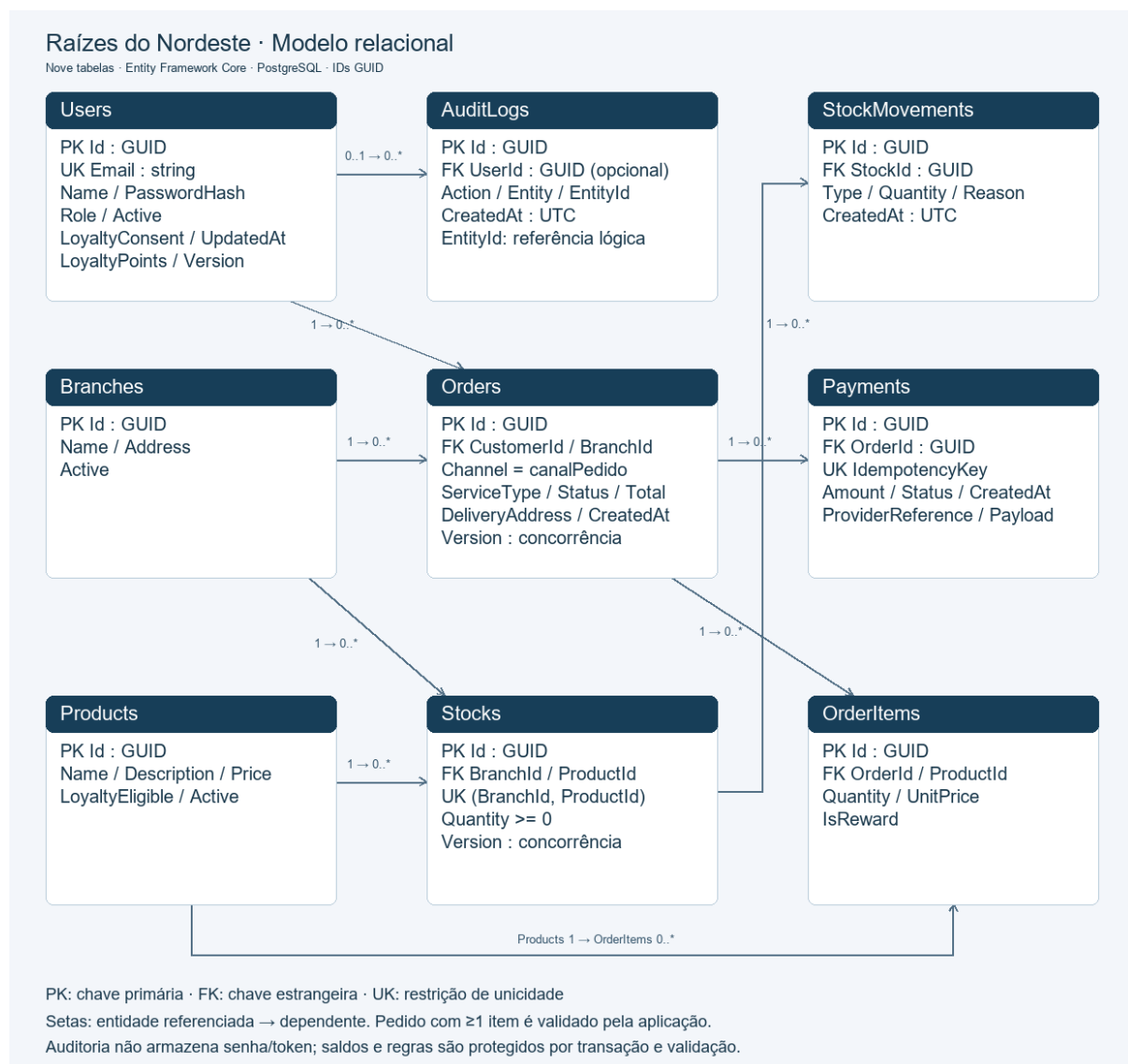


Figura 2 - DER: chaves primárias, estrangeiras e cardinalidades.

Restrições: e-mail, chave idempotente e par unidade/produto são únicos; saldos não podem ser negativos. AuditLogs.UserId é FK opcional; EntityId é referência lógica, sem FK. A aplicação exige ao menos um item por pedido. O contrato JSON expõe Channel como canalPedido. As migrations contêm as restrições completas.

2

Classes principais e regras de domínio

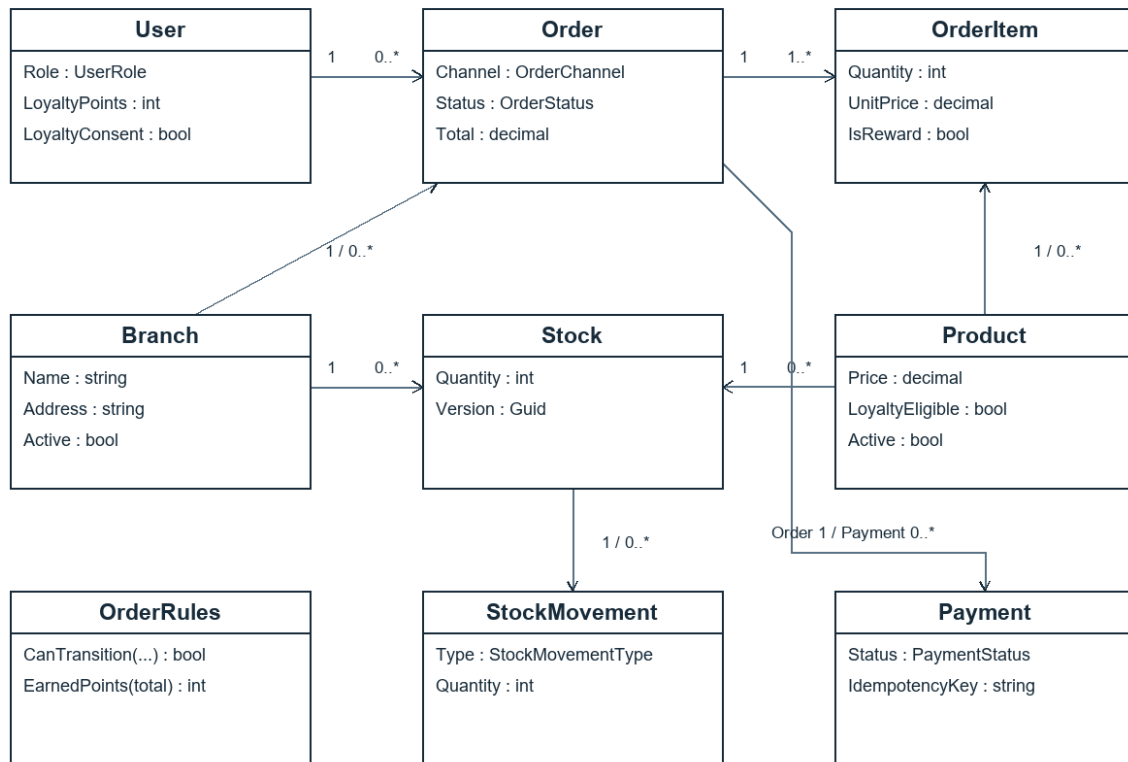


Figura 3 - Classes principais; atributos resumidos.

Order agrega os itens e pagamentos. Stock relaciona produto e unidade; StockMovement registra alterações de saldo. OrderRules define as transições e os pontos ganhos; OrderService aplica essas regras durante a transação. As entidades concentram estado e as regras ficam nos serviços/OrderRules. AuditLog, não detalhada na figura de classes, permanece representada no DER.

O preço unitário é copiado para OrderItem ao criar o pedido. Assim, uma edição posterior do catálogo não altera o total histórico. Desativar produto ou unidade evita novas utilizações sem apagar pedidos existentes. A cardinalidade mínima de um item por pedido é validada na aplicação.

2

Sequência - pedido e pagamento aprovado

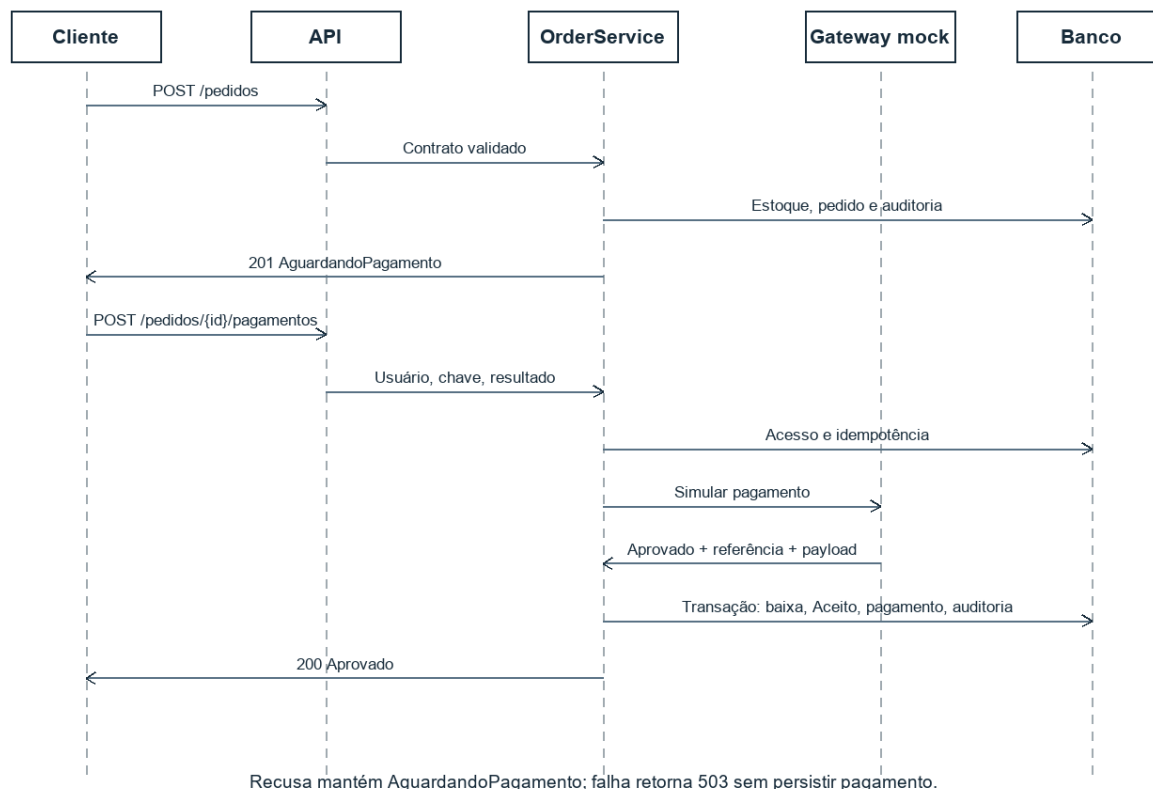


Figura 4 - Sequência do fluxo de criação e pagamento.

O retorno simplificado do serviço na figura passa pelo controller para chegar ao cliente. Pagamento e estoque são persistidos na mesma transação serializável. A idempotência é verificada após a autorização; a mesma chave não autoriza acesso de outro cliente nem pode ser reutilizada para um pedido diferente.

5 CONTRATOS REST E REGRAS OPERACIONAIS

O contrato completo e os exemplos de cada endpoint estão em docs/ENDPOINTS.md e docs/openapi.json. Swagger é servido em /swagger. IDs ilustrativos devem ser substituídos pelos IDs obtidos na API. Enums são representados como texto; valores numéricos enviados no JSON são recusados.

Método e rota	Acesso / sucesso
POST /auth/register	Público; 201
POST /auth/login	Público; 200
GET /unidades	Qualquer perfil autenticado.; 200
POST /unidades	Perfis: Administrador; 201
GET /unidades/{id}	Qualquer perfil autenticado.; 200
PUT /unidades/{id}	Perfis: Administrador; 200

DELETE /unidades/{id}	Perfis: Administrador; 204
GET /produtos	Qualquer perfil autenticado.; 200
POST /produtos	Perfis: Gerente,Administrador; 201
GET /produtos/{id}	Qualquer perfil autenticado.; 200
PUT /produtos/{id}	Perfis: Gerente,Administrador; 200
DELETE /produtos/{id}	Perfis: Gerente,Administrador; 204
GET /unidades/{branchId}/cardapio	Qualquer perfil autenticado.; 200
GET /estoque/{branchId}	Perfis: Atendente,Gerente,Administrador; 200
POST /estoque/{branchId}/entradas	Perfis: Gerente,Administrador; 200
POST /estoque/{branchId}/saidas	Perfis: Gerente,Administrador; 200
GET /estoque/{branchId}/movimentacoes	Perfis: Gerente,Administrador; 200
POST /pedidos	Perfis: Cliente,Atendente,Gerente,Administrador; 201
GET /pedidos	Qualquer perfil autenticado.; 200
GET /pedidos/{id}	Qualquer perfil autenticado.; 200
POST /pedidos/{id}/pagamentos	Perfis: Cliente,Atendente,Gerente,Administrador; 200
PATCH /pedidos/{id}/status	Perfis: Atendente,Cozinha,Entregador,Gerente,Administrador; 200
PATCH /pedidos/{id}/cancelar	Perfis: Cliente,Atendente,Gerente,Administrador; 200
GET /usuarios/me	Qualquer perfil autenticado.; 200
GET /fidelidade	Qualquer perfil autenticado.; 200
PATCH /fidelidade/consentimento	Qualquer perfil autenticado.; 200
GET /auditoria	Perfis: Gerente,Administrador; 200
GET /health	Público; 200

Paginação e códigos HTTP

Listagens usam page a partir de 1 e limit entre 1 e 100; retornam arrays com ordenação estável. 201 indica criação, 200 consulta/ação e 204 desativação sem corpo. Erros usam envelope único: error, message, details, timestamp, path e requestId. Contratos inválidos retornam 400; autenticação 401; autorização 403; ausência 404; conflito 409; regras de domínio 422; limite de auth 429; falha interna 500; indisponibilidade mock 503.

Exemplo de pedido

```
{
  "branchId": "00000000-0000-0000-0000-000000000001",
  "canalPedido": "TOTEM",
  "serviceType": "Retirada",
  "items": [
    {
      "productId": "00000000-0000-0000-0000-000000000002",
      "quantity": 1,
      "useReward": false
    }
  ]
}
```

O servidor calcula o preço pelo catálogo e verifica o estoque agregado, inclusive para itens repetidos. O pedido inicia em `AguardandoPagamento`. Entrega exige endereço; retirada e consumo local não precisam armazená-lo. O controle de concorrência impede saldo negativo.

Exemplo de pagamento

```
{
  "idempotencyKey": "pedido-exemplo-001",
  "approve": true,
  "simulateFailure": false
}
```

A resposta HTTP 200 inclui `id`, `orderId`, `amount`, `status`, `providerReference`, `providerPayload` e `createdAt`. Aprovado move o pedido a `Aceito` e baixa estoque; Recusado preserva `AguardandoPagamento`. `simulateFailure` retorna 503 sem gravar pagamento. Repetir a mesma chave com o mesmo resultado devolve o registro existente.

Transições, cancelamento e fidelidade

O fluxo é `Aceito` -> `EmPreparo` -> `Pronto` -> `Finalizado`. Entrega inclui `EmRota` antes da finalização. Cozinha prepara e marca pronto; Atendente finaliza retirada/consumo local; Entregador conduz a entrega; Gerente/Administrador conduzem transições válidas. Cliente não altera status operacional.

Cancelamento é permitido antes do preparo e devolve estoque baixado e pontos reservados, marcando o pagamento como `Estornado`. Com consentimento, a finalização concede um ponto por R\$ 10 pagos. Cada item elegível resgatado consome 10 pontos. Pedido pendente não expira automaticamente; essa rotina permanece como evolução.

6 SEGURANÇA E PRIVACIDADE

Este é um protótipo acadêmico. Os controles abaixo não representam certificação ou garantia de conformidade legal.

Dados e finalidade técnica

Dado	Finalidade	Controle aplicado
Nome/e-mail	Identificação e login	Retornados no cadastro/perfil próprio; não no JWT
Senha	Autenticação	Apenas hash com PasswordHasher/PBKDF2; hash excluído de JSON
Endereço	Entrega	Exigido só para Entrega; descartado do contrato para retirada/consumo local
Pedido e pagamento mock	Operação e rastreabilidade	Cliente só acessa o próprio pedido; perfis operacionais podem consultar
Pontos/consentimento	Fidelização opcional	Valor, data de alteração e evento de auditoria; revogação disponível
UserId/EntityId/ação/data	Auditoria	Consulta restrita a Gerente/Administrador; sem senha/token/corpo da requisição

Para a demonstração, use exclusivamente dados fictícios. O consentimento implementado refere-se apenas à fidelidade; não se presume consentimento genérico para qualquer finalidade.

Proposta acadêmica de bases e retenção

Como hipótese a validar pelo controlador: nome, contato, pedido e endereço necessário à entrega se relacionam à execução do contrato (art. 7º, V); a fidelidade opcional depende de consentimento específico (art. 7º, I). Auditoria exige definição da finalidade e base aplicável, não uma autorização genérica para guardar tudo. O término do tratamento e as exceções de conservação são tratados nos arts. 15 e 16; acesso, correção e demais direitos constam do art. 18. Esta proposta não é parecer jurídico. [LGPD, texto oficial](https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709compilado.htm).

No protótipo, a política proposta é eliminar dados fictícios ao encerrar a avaliação. Para uso real, o responsável deve aprovar uma tabela de retenção por finalidade, obrigações aplicáveis e prazos de backup; não há prazo universal definido neste projeto. A anonimização proposta remove nome, e-mail e endereço, impede reidentificação nas estatísticas e considera cópias de segurança. A rotina e o canal de atendimento ao titular permanecem como evolução documentada.

Implementado

JWT com assinatura HMAC SHA-256, emissor, público destinatário e validade de uma hora; verificação de assinatura e prazo em todas as rotas protegidas.

Cadastro público fixa perfil Cliente; privilégios operacionais não vêm do body enviado pelo usuário.

Autorização antes de consultar/retornar pagamento por chave de idempotência.

Validações de contrato, limitação de 30 requisições/minuto por IP no módulo auth e erros JSON sem stack traces.

Logs sem payloads e sem dados pessoais; auditoria de login, cadastro, consulta de perfil/pedido, estoque, pagamento, status, cancelamento e consentimento.

Preço calculado pelo servidor; transações serializáveis, tokens de concorrência e restrições de saldo não negativo.

Chave privada obrigatória fora de Development; .env e bancos locais excluídos do Git.

Limitações explícitas

Contas administrativas de demonstração são públicas. Qualquer pessoa com essas credenciais pode modificar dados fictícios. Desabilitar o seed não remove contas que já existem; não reutilize esse banco em produção.

Não há refresh/logout com revogação imediata, escopo de funcionário por unidade, trilha de consentimento com versão de política, gestão administrativa de usuários ou exclusão/anonimização automática.

Logs/auditoria não têm retenção automática. A proposta é anonimizar identificação e endereços quando cessar a finalidade e expirar o prazo aprovado pelo responsável, mantendo apenas estatísticas necessárias; a rotina não foi implementada.

O limitador é por processo/IP. Proxy reverso, múltiplas réplicas e encaminhamento confiável de IP exigem configuração específica no deploy.

TLS deve ser fornecido pela hospedagem; a porta HTTP local serve somente ao desenvolvimento. Não habilitar CORS irrestrito nem enviar dados de cartão.

Não há avaliação jurídica, teste de invasão ou teste de carga.

7 TESTES E RESULTADOS

Data local da revisão: **02/09/2026**.

Verificação	Resultado
dotnet build	Sucesso, zero erros e zero avisos antes da documentação
dotnet test	36 aprovados, zero falhas, incluindo testes de integração
PostgreSQL e migration	Conexão aprovada; has-pending-model-changes: nenhuma alteração pendente
/health	HTTP 200, status: ok
Swagger/OpenAPI	HTTP 200; 22 caminhos, 23 schemas, Bearer e exemplos

Login JWT	Cliente e perfis operacionais autenticados
Fluxo manual	Pedido criado, pagamento mock, preparo, pronto e finalização confirmados
Publicação Release	dotnet publish concluído sem erro
Docker Compose	Sintaxe validada com docker compose config --quiet; engine não iniciada
Repositório público	Link respondeu HTTP 200 sem autenticação; alterações finais ainda locais
Estoque/fidelidade/auditoria	Cobertos pela suíte de integração

Os testes de integração criam SQLite temporário isolado e verificam autenticação/autorização, contratos, pedidos, pagamento, idempotência, estoque, concorrência, cancelamento, fidelidade, auditoria, paginação e Swagger.

A coleção Postman foi executada no PostgreSQL: 44 requisições e 79 asserções aprovadas, com zero falhas. Evidência: docs/evidencias/postman-postgres.xml. Docker permanece opcional e não foi declarado como executado neste computador enquanto a virtualização estiver indisponível.

As verificações não representam teste de carga, pentest, certificação jurídica ou garantia de disponibilidade em produção.

Plano executável da coleção

Cenário	HTTP esperado
T01 - Login administrador	200
T02 - Criar unidade exclusiva da execução	201
T03 - Criar produto exclusivo	201
T04 - Entrada de estoque	200
T05 - Login cliente	200
T06 - Consultar cardápio por unidade	200
T07 - Criar pedido multicanal	201
T08 - Pagamento recusado	200
T09 - Pedido aguarda nova tentativa	200
T10 - Falha simulada do gateway	503
T11 - Pagamento aprovado	200
T12 - Repetir mesma chave sem cobrar novamente	200
T13 - Confirmar pedido aceito	200

T14 - Cliente não pode preparar pedido	403
T15 - Preparar	200
T16 - Pronto	200
T17 - Finalizar retirada	200
T18 - Consultar auditoria do pedido	200
T19 - Filtrar canal	200
T20 - Conferir baixa única no estoque	200
T21 - Sem token	401
T22 - Login inválido	401
T23 - Canal ausente	400
T24 - Canal inválido	400
T25 - Quantidade negativa	400
T26 - Unidade inexistente	404
T27 - Produto inexistente	404
T28 - Estoque insuficiente	409
T29 - Transição inválida	409
T30 - E-mail inválido	400
T31 - Criar pedido para cancelamento	201
T32 - Pagar pedido cancelável	200
T33 - Cancelar e estornar	200
T34 - Conferir devolução de estoque	200
T35 - Editar unidade	200
T36 - Editar produto	200
T37 - Saída manual por perda	200
T38 - Saída acima do saldo	409
T39 - Histórico de estoque	200
T40 - Desativar produto	204
T41 - Produto desativado indisponível	404
T42 - Desativar unidade	204

T43 - Unidade desativada indisponível	404
T44 - Histórico do pedido preservado	200

Pré-condição: API com seed demonstrativo. Executar todas as pastas na ordem, pois os tokens e IDs são capturados pelas requisições anteriores. Cada execução cria sua própria unidade e produto. A coleção verifica também o formato dos erros e efeitos de negócio; entradas, asserções e resultados ficam nos arquivos JSON/XML do repositório.

8 EXECUÇÃO E ENTREGA

Pré-requisito mínimo: .NET SDK 10. Na pasta que contém a solução:

```
dotnet restore
dotnet run --project RaizesDoNordeste.API/RaizesDoNordeste.API.csproj
```

Acesse <http://localhost:5287/swagger>. O PostgreSQL deve estar ativo; a API aplica a migration e o seed. Use as contas fictícias do README. Docker, Postman e Node.js são opcionais. Postman ou Newman permitem executar a coleção completa; dotnet test executa os testes automatizados.

Para PostgreSQL instalado, crie um database vazio e configure ConnectionStrings:Default, Jwt:Key e Demo:Seed por User Secrets, conforme o README. Para Docker operacional: gere .env e execute docker compose up --build -d. A API espera a saúde do PostgreSQL e o banco persiste em volume separado.

O repositório deve ser público, conter código, README, coleção e documentação, e não incluir segredos, bancos locais, bin/obj ou logs. O avaliador pode clonar e executar o README. GitHub Pages não executa a API .NET; hospedagem online, Codespaces e Render são opcionais. Os integrantes farão os commits e o push e verificarão a primeira execução da CI.

9 CONCLUSÃO E EVOLUÇÕES

O fluxo A está implementado e validado no banco relacional de teste e no fluxo manual PostgreSQL: pedido multicanal, pagamento mock, estoque, status e fidelidade possuem regras verificáveis e trilha de auditoria. As migrations e o seed tornam a inicialização reproduzível; a coleção demonstra o comportamento de sucesso e os principais erros.

As evidências são delimitadas ao ambiente e aos cenários executados. Não demonstram capacidade em produção, conformidade legal ou execução Docker. A finalização administrativa depende da identificação do relatório, da revisão dos integrantes e da publicação do repositório. Não foi criado histórico fictício de commits.

Campanha teria unidade/canais elegíveis, início/fim em UTC, desconto percentual limitado e flag ativa. Aplicação verificaria vigência e elegibilidade antes de calcular total; guardaria no pedido o desconto aplicado, sem recalculando pedidos antigos. Não acumularia desconto de campanha com item gratuito da fidelidade. Uma futura implementação precisaria de tabela, migrations, testes de limites e documentação de novos contratos.

Outras evoluções: vínculo de funcionários à unidade, gestão administrativa de usuários, revogação imediata de tokens, expiração de pedidos pendentes, rotina de anonimização, observabilidade e testes de carga. Essas propostas não são apresentadas como funcionalidades existentes.

REFERÊNCIAS

ROTEIRO DE ATIVIDADE PRÁTICA. Projeto Back End, 2026. Documento fornecido para especificação do projeto acadêmico.